SORBONNE UNIVERSITY
Master 2 IMA
Department of Computer Science

Report IG3DA Project

Online Neural Denoising with Cross-Regression for Interactive Rendering

Prepared by:

Ko Tchong Sem ALPOU

Supervisor
Jonathan Fabrizio

Contents

1 Introduction **1**

2 State of the art **1**

3 Technical Approach **1**

3.1 Cross-Regression for Pilot Estimates 2
3.2 Denoising Neural Network Architecture 2
3.3 Adaptive Spatiotemporal Filtering 2
3.4 Online Self-Supervised Learning 2
3.5 Imlementation details 3
3.5.1 Spatiotemporal filter 3
3.5.2 Temporal Reprojection 3

4 Evaluation and Results **3**

4.1 Performance Metrics 3
4.2 Quantitative and Qualitative Analysis 4
4.3 Implementation Limitations 5

5 Conclusion and Future Work **5**

1 Introduction

The primary goal of **Monte Carlo (MC) ray tracing** is to synthesize various lighting effects by simulating the behavior of randomly casted light paths. This process generates unbiased pixel estimates that converge toward a ground truth image as more samples are taken. If render time is unconstrained, a high number of samples per pixel (spp) ensures smooth results.
However, in interactive rendering scenarios, the sample size is strictly limited (e.g 4 spp) to maintain performance, which inevitably results in very noisy images. To address this limitation, the idea is to rapidly generate noisy images with few sample of rays and then launch a whole process of denoising these images to make them look more realistic.
This report details a hybrid denoising framework (1) that combines the strengths of classical and neural approaches. The method utilizes a modified local regression (**cross-regression**), to generate "pilot estimates" from noisy images. These estimtes serve as the input for a denoising neural network that learns to reduce noise and preserve complex edges.

2 State of the art

- **Filters with G-buffers:** The methods use noise-free geometric buffers (G-buffers), such as texture and normals to guide the denoising process. This ensures that image edges are not blurred.
  - **SVGF (Spatiotemporal Variance-Guided Filtering) (4):** A cross-bilateral filter that optimizes denoising parameters using pixel color variance.
  - **Edge-Avoiding À-trous Filters (2):** Hierarchical filters used to reduce noise while respecting geometric edges.
- **Local Regression Methods:** These approximate ground truth image functions by fitting local linear models to observed noisy samples within a denoising window.
  - **BMFR (Blockwise Multi-Order Feature Regression) (3):** n efficient regression-based method that uses G-buffers as explanatory variables for real-time path-tracing reconstruction.
- **Major Limitation:** While effective at preserving geometric edges, these methods tend to blur non-geometric details, such as shadows, because G-buffers cannot capture high-frequency illumination changes.

3 Technical Approach

The core of this framework is a hybrid architecture that integrates classical local regression with a deep neural network optimized through online learning. This combination allows for sharp, real-time denoising without the need for an external pre-training dataset.

3.1 Cross-Regression for Pilot Estimates

The pipeline begins by splitting the color samples produced by the path tracer into two disjoint buffers, y^A and y^B, each providing an independent but noisy estimate of the ground truth. To overcome the blurring limitations of standard regression, the method introduces cross-regression.

$$x_i^A - x_c^A = \left[\frac{y_i^A - y_c^A}{\hat{\sigma}_i^A + \hat{\sigma}_c^A + \epsilon}, \rho_i - \rho_c, n_i - n_c \right]^T$$

ρ represents textures and n represents normals.

A linear function is formed using the features of one buffer to fit the noisy colors of the other. Specifically, regression parameters from buffer A are used to predict buffer B, and vice versa. This process generates two pilot estimates, $\tilde{f}(x^A)$ and $\tilde{f}(x^B)$, which reduce variance while preserving non-geometric edges, such as shadows, that G-buffers alone cannot capture.

3.2 Denoising Neural Network Architecture

The framework utilizes a simplified U-Net architecture containing approximately 30,000 learnable parameters. It takes as input the two pilot estimates, the G-buffers (textures and normals), and the reprojected denoising output from the previous frame, $\mathcal{R}\hat{f}^P(x)$.

Rather than predicting pixel colors directly, the network infers six per-pixel parameters, $\theta_c = [\theta_c^A, \theta_c^B, \theta_c^\rho, \theta_c^n, \theta_c^p, \theta_c^\alpha]$. These parameters act as adaptive weights for a subsequent spatiotemporal filter, defining the sensitivity to color changes, geometric features, and temporal blending.

3.3 Adaptive Spatiotemporal Filtering

The final denoising is performed by an adaptive filter that processes each pixel within an 11×11 window:

- Spatial Filtering: The filter applies cross-bilateral weighting where the weights m_i are determined by the network-predicted parameters θ_c.
- Temporal Blending: The resulting spatial average is blended with the reprojected previous frame using a blending parameter θ_c^α:

$$\hat{f}(x_c^A) = \theta_c^\alpha \left(\frac{\sum m_i^A \tilde{f}(x_i^A)}{\sum m_i^A} \right) + (1 - \theta_c^\alpha)\mathcal{R}\hat{f}^P(x_c)$$

Final Fusion: The two resulting denoised estimates, $\hat{f}(x^A)$ and $\hat{f}(x^B)$, are combined into the final output image $\hat{f}(x)$

3.4 Online Self-Supervised Learning

The most significant innovation of this approach is the elimination of ground-truth data requirements. The network learns in real-time at each frame through a self-supervised loss $\mathcal{L}$:

- Spatial Loss ($\mathcal{L}_s$): Measures the discrepancy between a denoising output from one buffer and the pilot estimate of the other to prevent the network from overfitting to noise.
- Temporal Loss ($\mathcal{L}_t$): Compares current denoising outputs with reprojected pilot estimates from the previous frame to ensure visual stability.

The network parameters are updated per frame using a single gradient step via the Adam optimizer. This allow the model to adapt dynamically to the specific image sequence being rendered at runtime.

3.5 Imlementation details

3.5.1 Spatiotemporal filter

To tackle the huge VRAM demands of processing high-resolution imagery, we implementated a tiling strategy that divides the frame into 128 X 128 blocks. This approach prevents the memory explosion which multiply VRAM usage by the window size (11 X 11).
The numerical stability is maintained by applying $softplus$ and $sigmoid$ functions to the network's output parameters (θ_c), to ensure that the variances remain positive and the temporal blending factor stays within $[0, 1]$.
Furthermore, image reconstruction from these patches is achieved using a reflect padding mode and a precise center crop to eliminate pixel shifts between tiles. For a padded input tensor T_{pad} with dimensions $(B, C, H+2p, W+2p)$, where p is the padding size, the center crop is defined as: $T_{crop} = T_{pad}[:, :, p : H + p, p : W + p]$.

3.5.2 Temporal Reprojection

Temporal reprojection, denoted as R, is executed at each new frame to exploit the temporal coherence between successive images. Its primary objective is to suppress flickering and accumulate information over time by reusing data from the "history buffer".
The process effectively finds the corresponding position of a current pixel p_n in the previous frame p_{n-1} by applying the principle of inverse reprojection. Mathematically, this transformation is governed by the view (V) and projection (P) matrices of the current and previous frames:

$$p_{n-1} = P_{n-1} \cdot V_{n-1} \cdot V_n^{-1} \cdot P_n^{-1} \cdot p_n$$

In the implementation, we first use the motion vectors (available the vmec file) applied to a normalized coordinate grid ($sampling_grid = base_grid + mv$). The we perform a bilinear interpolation to generate the warped image $R\hat{f}^P(x)$.

4 Evaluation and Results

4.1 Performance Metrics

- **PSNR (Peak Signal-to-Noise Ratio)**: This metric quantifies the reconstruction quality of the images. It is calculated as $PSNR = 10 \cdot \log_{10}(\frac{d^2}{MSE})$, where d is the

maximum possible pixel value and MSE is the Mean Squared Error.

- **SSIM (Structural Similarity Index)**: Rather than just measuring pixel differences, SSIM assesses the structural similarity between the denoised image and the reference. It considers luminance, contrast, and structure through the formula: $SSIM(x,y) = \frac{(2\mu_x\mu_y+c_1)(2\sigma_{xy}+c_2)}{(\mu_x^2+\mu_y^2+c_1)(\sigma_x^2+\sigma_y^2+c_2)}$

4.2 Quantitative and Qualitative Analysis

The training process shows strong performance convergence after 100 iterations.

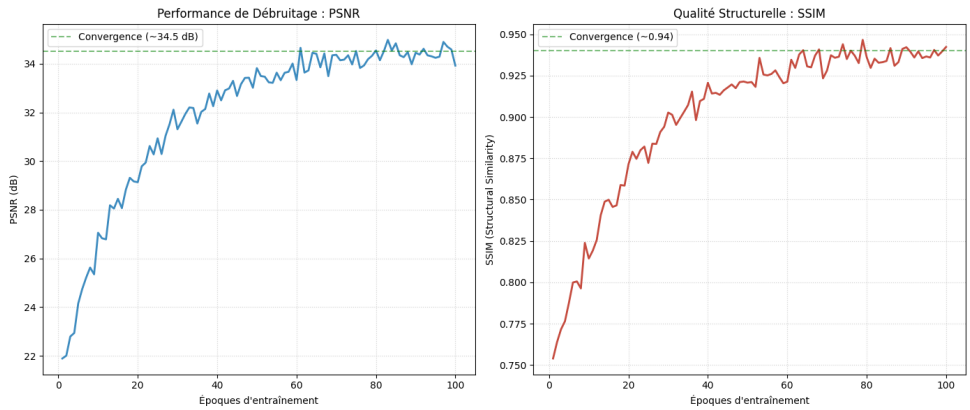

Figure 1: Evolution of the metrics

The system reaches a convergence point of approximately 34.5 dB, indicating a high level of signal recovery relative to the noise floor.
The structural quality converges at approximately 0.94, showing that the denoiser preserves edges and fine details effectively.

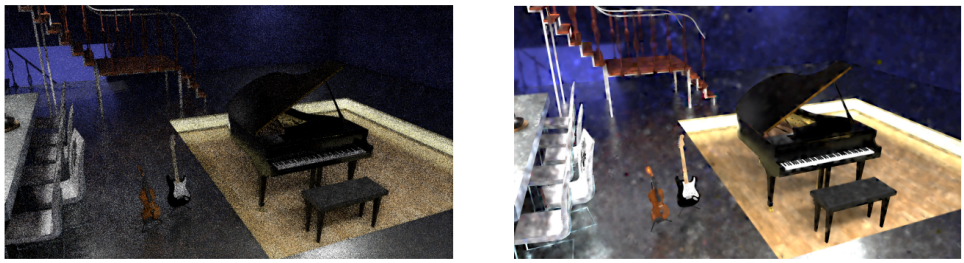

Figure 2: Qualitative results

The results are described as "relatively flat" during the final stages of training, which indicates that the temporal instability (flickering) common in interactive rendering has been successfully resolved.

4.3 Implementation Limitations

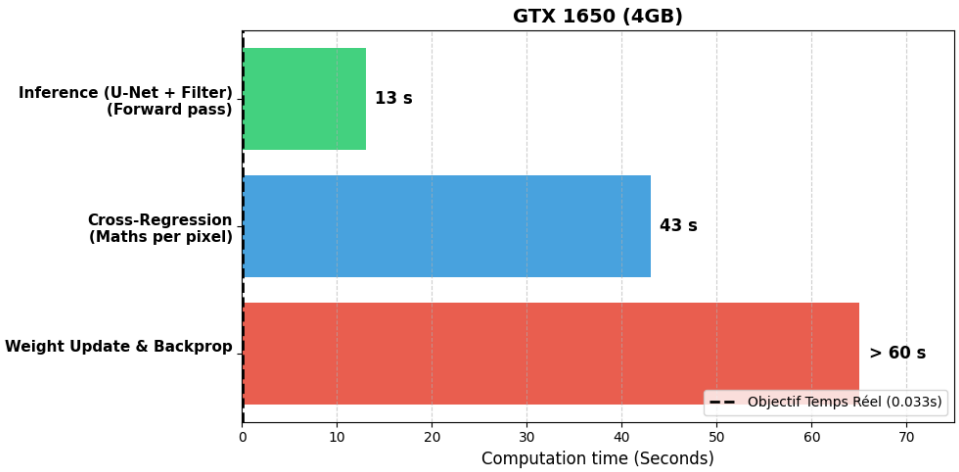

Figure 3: Computation time

While the denoising quality is high, tests on a GTX 1650 (4GB) reveal significant performance bottlenecks relative to the real-time target of 0.033s (60 FPS). These results suggest that while the mathematical model is robust, further optimization

5 Conclusion and Future Work

This project implemented a hybrid denoising framework that bridges the gap between classical filters and deep learning. By utilizing online learning, the system suppresses extreme Monte Carlo noise in real-time without external ground truth datasets Edge Preservation: Cross-regression pilot estimates allow the model to preserve non-geometric edges, such as complex shadows, which are lost in traditional G-buffer filtering.
Self-Supervised Stability: By splitting input into disjoint buffers and applying spatial and temporal losses, the network extracts common signals while ignoring independent noise.
Resource Management: A tiling strategy and Cholesky decomposition addressed VRAM bottlenecks and improved numerical stability.

References

[1] Choi, H., Hong, S., Ha, I., Kang, N. and Moon, B. (2024), 'Online neural denoising with cross-regression for interactive rendering', *ACM Transactions on Graphics (TOG)* **43**(6), 1–12.

[2] Dammertz, H., Sewtz, D., Hanika, J. and Lensch, H. P. (2010), Edge-avoiding a-trous wavelet transform for fast global illumination filtering, *in* 'Proceedings of the Conference on High Performance Graphics', pp. 67–75.

[3] Koskela, M., Immonen, K., Mäkitalo, M., Foi, A., Viitanen, T., Jääskeläinen, P., Kultala, H. and Takala, J. (2019), 'Blockwise multi-order feature regression for real-

time path-tracing reconstruction', *ACM Transactions on Graphics (TOG)* **38**(5), 1–14.

[4] Schied, C., Kaplanyan, A., Wyman, C., Patney, A., Chaitanya, C. R. A., Burgess, J., Liu, S., Dachsbacher, C., Lefohn, A. and Salvi, M. (2017), Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination, *in* 'Proceedings of high performance graphics', pp. 1–12.